



# CI1056 – ALGORITMOS E ESTRUTURA DE DADOS II

Profa. Rachel Reis  
[rachel@inf.ufpr.br](mailto:rachel@inf.ufpr.br)



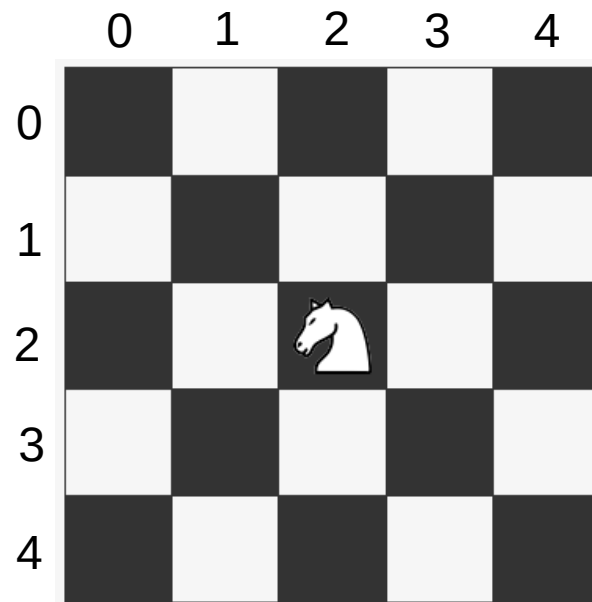
# AULA DE HOJE

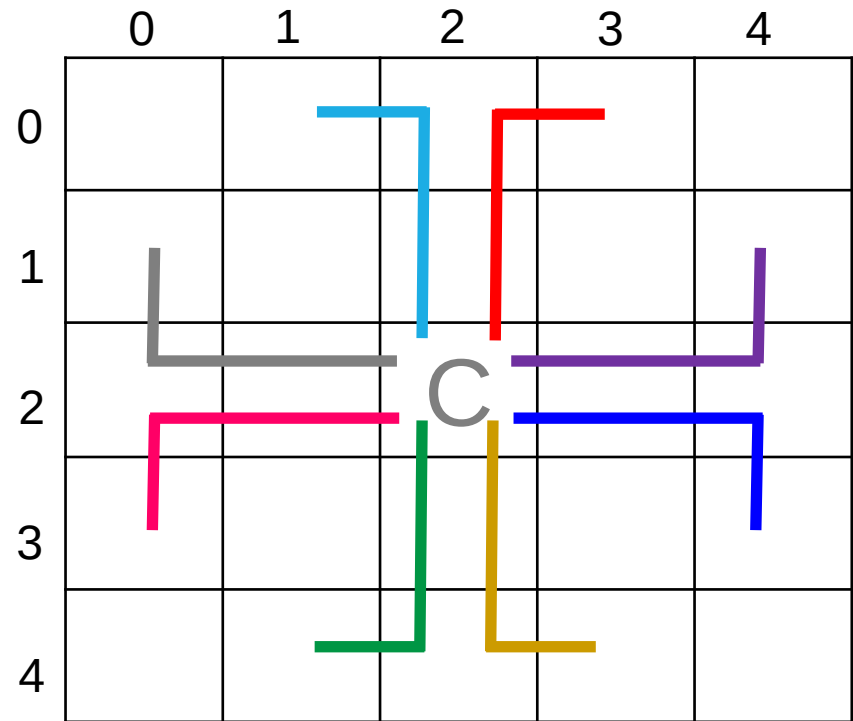
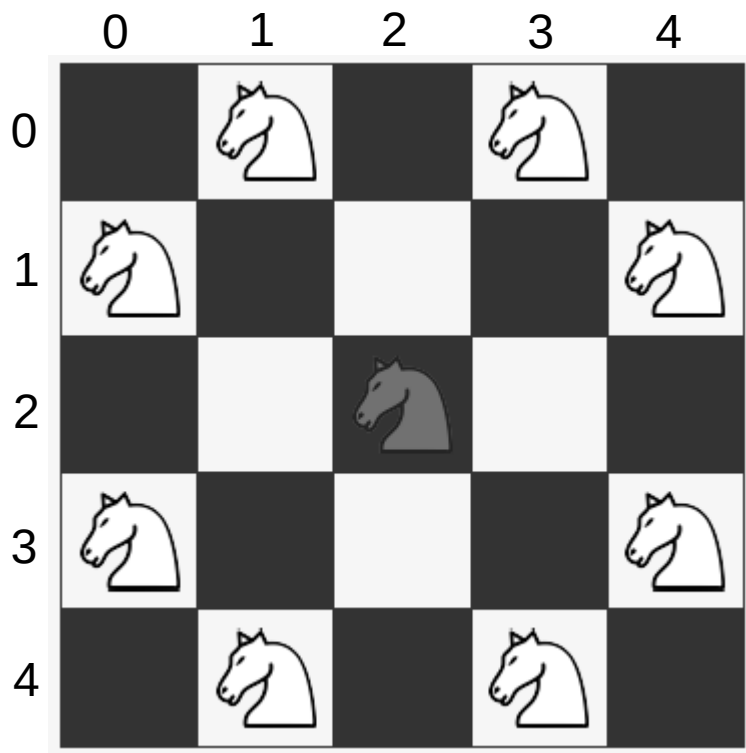
## Backtracking e o Passeio do Cavalo

Adaptação do material gentilmente  
disponibilizado pelo Prof. André Vignatti

# PASSEIO DO CAVALO

- Dado um tabuleiro de xadrez  $n \times n$  e uma posição  $(x, y)$ , será que é possível o cavalo percorrer **todo** o tabuleiro, tal que todas as casas sejam visitadas uma única vez?

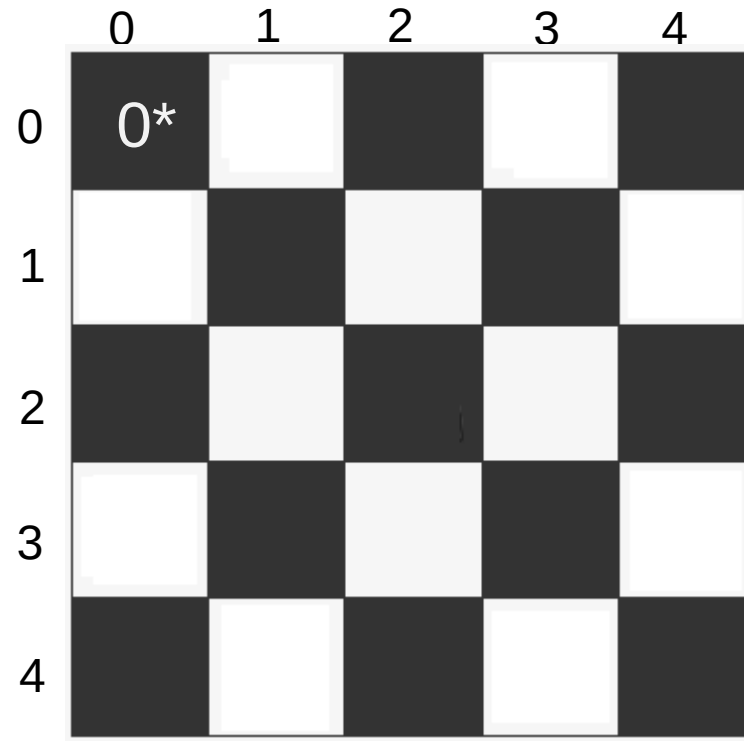




- Cavalos se movimentam no formato de L
- Movimentos do cavalo no tabuleiro:  
 $\text{eixoX} \rightarrow \{-2, -2, -1, 1, 2, 2, 1, -1\}$   
 $\text{eixoY} \rightarrow \{-1, 1, 2, 2, 1, -1, -2, -2\}$

# PROBLEMA – PASSEIO DO CAVALO

- Buscando uma solução.



Posição atual\*: 0, 0

# PROBLEMA – PASSEIO DO CAVALO

- Chegamos em um beco sem saída.

Como proceder?

|   | 0 | 1   | 2  | 3  | 4  |
|---|---|-----|----|----|----|
| 0 | 0 |     |    | 9  |    |
| 1 |   |     | 1  |    |    |
| 2 | 2 | 13  | 10 | 5  | 8  |
| 3 |   | 6   | 3  | 14 | 11 |
| 4 |   | 15* | 12 | 7  | 4  |

Posição atual\*: 4, 1

# PROBLEMA – PASSEIO DO CAVALO

- Retrocedendo...

Como proceder?



Passo backtrack  
(remover o movimento 15)

|   | 0 | 1  | 2  | 3   | 4  |
|---|---|----|----|-----|----|
| 0 | 0 |    |    | 9   |    |
| 1 |   |    | 1  |     |    |
| 2 | 2 | 13 | 10 | 5   | 8  |
| 3 |   | 6  | 3  | 14* | 11 |
| 4 |   | 15 | 12 | 7   | 4  |

Posição atual\*: 3, 3

eixoX  $\rightarrow$   $\{-2, -2, -1, 1, 2, 2, 1, -1\}$   
eixoY  $\rightarrow$   $\{-1, 1, 2, 2, 1, -1, -2, -2\}$



# PROBLEMA – PASSEIO DO CAVALO

- Buscando um novo caminho.

Como proceder?



Passo backtrack  
(remover o movimento 15)



Existe outro movimento  
a partir da casa 14?

|   | 0 | 1  | 2  | 3   | 4  |
|---|---|----|----|-----|----|
| 0 | 0 |    |    | 9   |    |
| 1 |   |    | 1  |     | 15 |
| 2 | 2 | 13 | 10 | 5   | 8  |
| 3 |   | 6  | 3  | 14* | 11 |
| 4 |   |    | 12 | 7   | 4  |

Posição atual\*: 3, 3  
Nova posição: 1, 4

# PROBLEMA – PASSEIO DO CAVALO

- Chegamos novamente em um beco sem saída.

Como proceder?

|   | 0   | 1  | 2  | 3  | 4  |
|---|-----|----|----|----|----|
| 0 | 0   |    | 16 | 9  |    |
| 1 | 17* |    | 1  |    | 15 |
| 2 | 2   | 13 | 10 | 5  | 8  |
| 3 |     | 6  | 3  | 14 | 11 |
| 4 |     |    | 12 | 7  | 4  |

Posição atual\*: 1, 0

# PROBLEMA – PASSEIO DO CAVALO

- Retrocedendo...

Como proceder?



Passo backtrack  
(remove o movimento 17)

|   | 0             | 1  | 2   | 3  | 4  |
|---|---------------|----|-----|----|----|
| 0 | 0             |    | 16* | 9  |    |
| 1 | <del>17</del> |    | 1   |    | 15 |
| 2 | 2             | 13 | 10  | 5  | 8  |
| 3 |               | 6  | 3   | 14 | 11 |
| 4 |               |    | 12  | 7  | 4  |

Posição atual\*: 0, 2

# PROBLEMA – PASSEIO DO CAVALO

- Buscando um novo caminho.

Como proceder?



Passo backtrack  
(remove o movimento 17)



Existe outro movimento a  
partir da casa 16?

|   | 0 | 1  | 2   | 3  | 4  |
|---|---|----|-----|----|----|
| 0 | 0 |    | 16* | 9  |    |
| 1 | × |    | 1   |    | 15 |
| 2 | 2 | 13 | 10  | 5  | 8  |
| 3 |   | 6  | 3   | 14 | 11 |
| 4 |   |    | 12  | 7  | 4  |

Posição atual\*: 0, 2

# PROBLEMA – PASSEIO DO CAVALO

- Mais uma vez chegamos em um beco sem saída.

Como proceder?

|   | 0 | 1  | 2   | 3  | 4  |
|---|---|----|-----|----|----|
| 0 | 0 |    | 16* | 9  |    |
| 1 |   |    | 1   |    | 15 |
| 2 | 2 | 13 | 10  | 5  | 8  |
| 3 |   | 6  | 3   | 14 | 11 |
| 4 |   |    | 12  | 7  | 4  |

Posição atual\*: 0, 2

# PROBLEMA – PASSEIO DO CAVALO

- Retrocedendo...

Como proceder?



Passo backtrack  
(remove o movimento 16)

|   | 0 | 1  | 2  | 3  | 4   |
|---|---|----|----|----|-----|
| 0 | 0 |    | 16 | 9  |     |
| 1 |   |    | 1  |    | 15* |
| 2 | 2 | 13 | 10 | 5  | 8   |
| 3 |   | 6  | 3  | 14 | 11  |
| 4 |   |    | 12 | 7  | 4   |

Posição atual\*: 1, 4

# PROBLEMA – PASSEIO DO CAVALO

- Buscando um novo caminho.

Como proceder?



Passo backtrack  
(remove o movimento 16)



Existe outro movimento a  
partir da casa 15?

|   | 0 | 1  | 2  | 3  | 4   |
|---|---|----|----|----|-----|
| 0 | 0 |    | ×  | 9  |     |
| 1 |   |    | 1  |    | 15* |
| 2 | 2 | 13 | 10 | 5  | 8   |
| 3 |   | 6  | 3  | 14 | 11  |
| 4 |   |    | 12 | 7  | 4   |

Posição atual\*: 1, 4

# PROBLEMA – PASSEIO DO CAVALO

- Outro beco sem saída ☹️

Como proceder?

|   | 0 | 1  | 2  | 3  | 4   |
|---|---|----|----|----|-----|
| 0 | 0 |    | ×  | 9  |     |
| 1 |   |    | 1  |    | 15* |
| 2 | 2 | 13 | 10 | 5  | 8   |
| 3 |   | 6  | 3  | 14 | 11  |
| 4 |   |    | 12 | 7  | 4   |

Posição atual\*: 1, 4

# PROBLEMA – PASSEIO DO CAVALO

- Retrocedendo...

Como proceder?



Passo backtrack  
(remove o movimento 15)

|   | 0 | 1  | 2  | 3   | 4             |
|---|---|----|----|-----|---------------|
| 0 | 0 |    |    | 9   |               |
| 1 |   |    | 1  |     | <del>15</del> |
| 2 | 2 | 13 | 10 | 5   | 8             |
| 3 |   | 6  | 3  | 14* | 11            |
| 4 |   |    | 12 | 7   | 4             |

Posição atual\*: 3, 3

# PROBLEMA – PASSEIO DO CAVALO

- Buscando um novo caminho.

Como proceder?



Passo backtrack  
(remove o movimento 15)



Existe outro movimento a  
partir da casa 14?

|   | 0 | 1  | 2  | 3   | 4  |
|---|---|----|----|-----|----|
| 0 | 0 |    |    | 9   |    |
| 1 |   |    | 1  |     | ×  |
| 2 | 2 | 13 | 10 | 5   | 8  |
| 3 |   | 6  | 3  | 14* | 11 |
| 4 |   | ×  | 12 | 7   | 4  |

Posição atual\*: 3, 3

# PROBLEMA – PASSEIO DO CAVALO

- Outro beco sem saída ☹️

Como proceder?

|   | 0 | 1  | 2  | 3   | 4  |
|---|---|----|----|-----|----|
| 0 | 0 |    |    | 9   |    |
| 1 |   |    | 1  |     | ×  |
| 2 | 2 | 13 | 10 | 5   | 8  |
| 3 |   | 6  | 3  | 14* | 11 |
| 4 |   | ×  | 12 | 7   | 4  |

Posição atual\*: 3, 3

# PROBLEMA – PASSEIO DO CAVALO

- Retrocedendo...

Como proceder?



Passo backtrack  
(remove o movimento 14)

|   | 0 | 1  | 2  | 3  | 4  |
|---|---|----|----|----|----|
| 0 | 0 |    |    | 9  |    |
| 1 |   |    | 1  |    |    |
| 2 | 2 | 13 | 10 | 5  | 8  |
| 3 |   | 6  | 3  | 14 | 11 |
| 4 |   |    | 12 | 7  | 4  |

Posição atual\*: 3, 3

# PROBLEMA – PASSEIO DO CAVALO

- Buscando um novo caminho.

Como proceder?



Passo backtrack  
(remove o movimento 14)



Existe outro movimento a partir da casa 13?

|   | 0 | 1   | 2  | 3 | 4  |
|---|---|-----|----|---|----|
| 0 | 0 |     |    | 9 |    |
| 1 |   |     | 1  |   |    |
| 2 | 2 | 13* | 10 | 5 | 8  |
| 3 |   | 6   | 3  |   | 11 |
| 4 |   |     | 12 | 7 | 4  |

Posição atual\*: 2, 1

# PROBLEMA – PASSEIO DO CAVALO

- Buscando um novo caminho.

Como proceder?

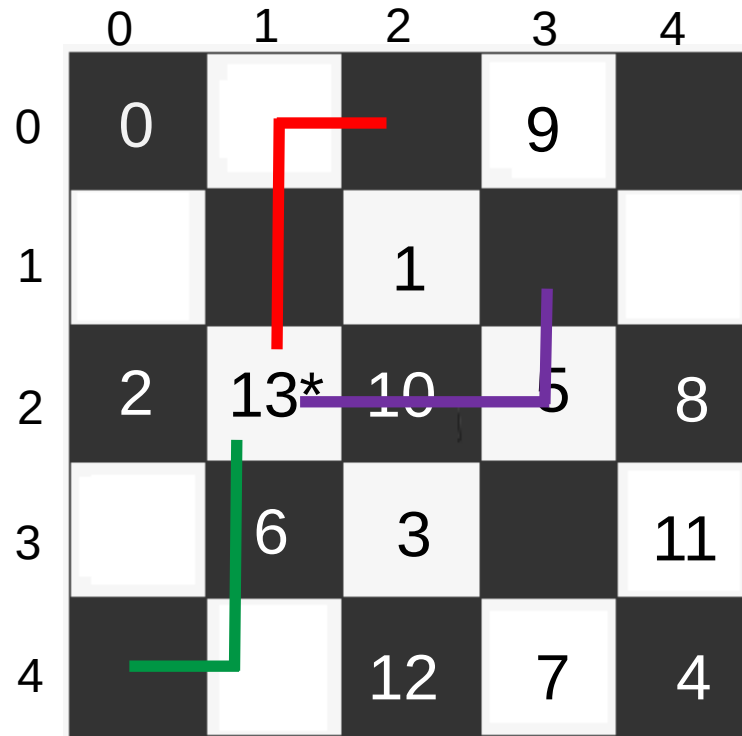


Passo backtrack  
(remove o movimento 14)



Existe outro movimento a partir da casa 13?

Continuar...



Posição atual\*: 2, 1

# BACKTRACKING FRAMEWORK GERAL

- Buscar todas as sequências  $x_1, x_2, \dots, x_n$  tal que uma propriedade  $P_n(x_1, x_2, \dots, x_n)$  é verdadeira.
  - Onde  $x_n$  pertence a um domínio  $D_k$  de inteiros.
- Exemplo: problema do passeio do cavalo
  - $x_1, x_2, \dots, x_n$ : cada  $x$  representa um movimento do cavalo no tabuleiro.
  - domínio  $D_k$  : definido no intervalo  $[0, 24]$ , representando as 25 casas do tabuleiro.
  - $P_n(x_1, x_2, \dots, x_n)$ : o cavalo só pode visitar uma casa do tabuleiro uma única vez.

# BACKTRACKING

- Exemplo de solução produtiva.

|    |    |    |    |    |
|----|----|----|----|----|
| 0  | 5  | 14 | 9  | 20 |
| 13 | 8  | 19 | 4  | 15 |
| 18 | 1  | 6  | 21 | 10 |
| 7  | 12 | 23 | 16 | 3  |
| 24 | 17 | 2  | 11 | 22 |

# BACKTRACKING IMPLEMENTAÇÃO

- Funções principais

- main()

|                        |
|------------------------|
| main()                 |
| <code>solveKT()</code> |

- solveKT

|                                             |
|---------------------------------------------|
| <code>solveKT()</code>                      |
| ...<br><code>solveKTUtil(...)</code><br>... |

- solveKTUtil

|                                       |
|---------------------------------------|
| <code>solveKTUtil(...)</code>         |
| ...<br><i>// backtracking</i><br>.... |

# BACKTRACKING IMPLEMENTAÇÃO

- Funções auxiliares

Imprime solução produtiva

|                    |
|--------------------|
| printSolution(...) |
| ...                |

Analisa se o movimento  
do cavalo é seguro

|             |
|-------------|
| isSafe(...) |
| ...         |

# IMPLEMENTAÇÃO

- Sobre a função solveKT
  - Exibe apenas a primeira solução produtiva (se existir).
  - Passos
    - Criar uma matriz  $n \times n$
    - Inicializar todas as casas da matriz com -1
    - Colocar o cavalo na posição (0,0)
    - Definir os movimentos do cavalo no tabuleiro
    - Chamar a função solveKTUtil (*backtracking*)

```
#include <stdio.h>
#define N 5

void solveKT(){
    int sol[N][N];

    for (int x = 0; x < N; x++)
        for (int y = 0; y < N; y++)
            sol[x][y] = -1;

    sol[0][0] = 0;

    int xMove[8] = { 2, 1, -1, -2, -2, -1, 1, 2 };
    int yMove[8] = { 1, 2, 2, 1, -1, -2, -2, -1 };

    if (solveKTUtil(0, 0, 1, sol, xMove, yMove) == 1)
        printSolution(sol);
    else
        printf("Solucao nao existe");
}
```

```
void printSolution(int sol[N][N])
{
    for (int x = 0; x < N; x++) {
        for (int y = 0; y < N; y++)
            printf(" %2d ", sol[x][y]);
        printf("\n");
    }
}
```

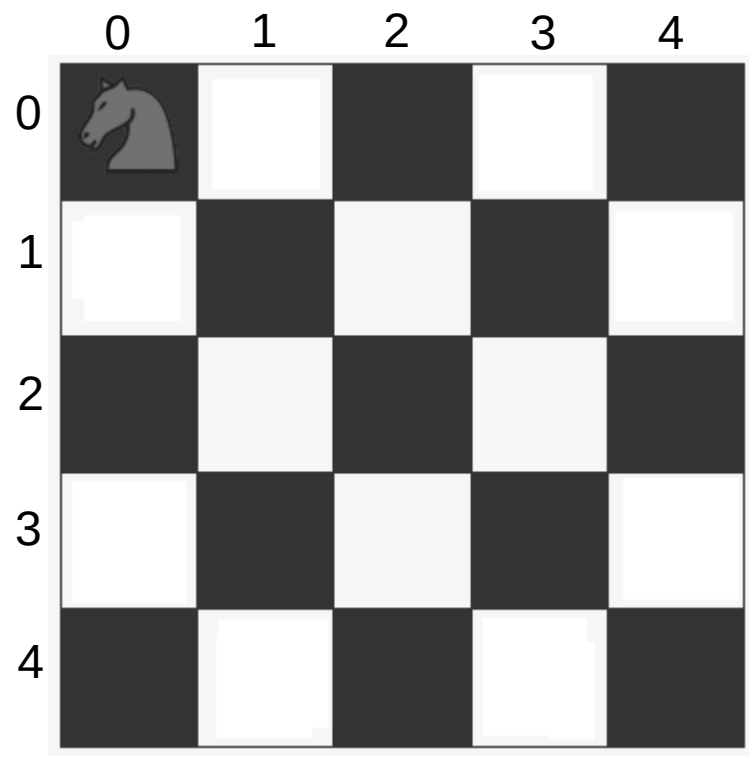
# IMPLEMENTAÇÃO

- Sobre a função solveKTUtil
  - Retorno: 1 se achou uma solução e 0 caso contrário
  - Parâmetros
    - x e y: posição atual do cavalo no tabuleiro.
    - movei: número de movimentos do cavalo [1, 24].
    - sol: matriz representando o tabuleiro de xadrez
    - xMove e yMove: pares de movimento do cavalo no tabuleiro.

```
int solveKTUtil(int x, int y, int movei, int sol[N][N], int xMove[N], int yMove[N])  
{
```



Posição atual do cavalo.  
Inicia com x=0 e y=0

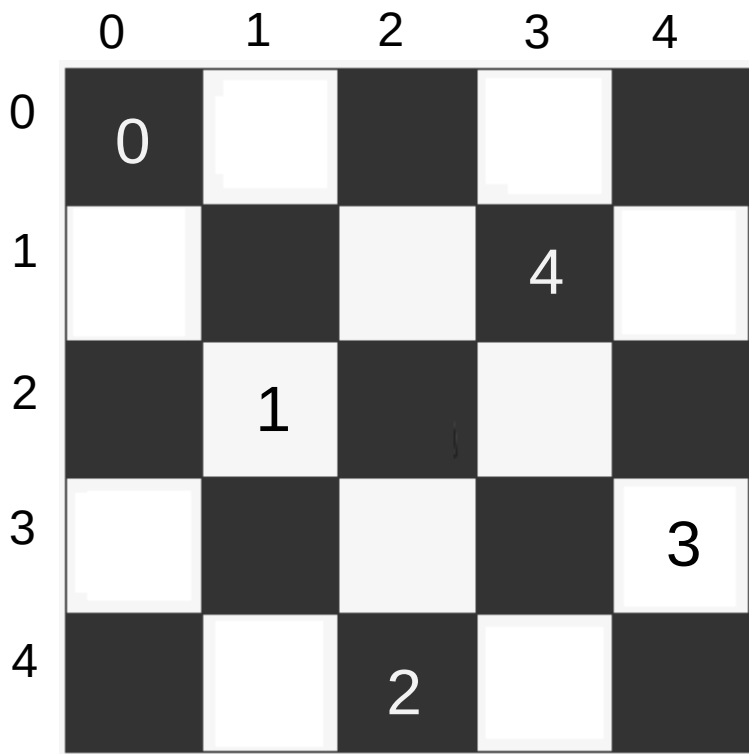


```
}
```

```
int solveKTUtil(int x, int y, int movei, int sol[N][N], int xMove[N], int yMove[N])  
{
```

Número de movimentos  
do cavalo. Inicia com movei=1.

Matriz

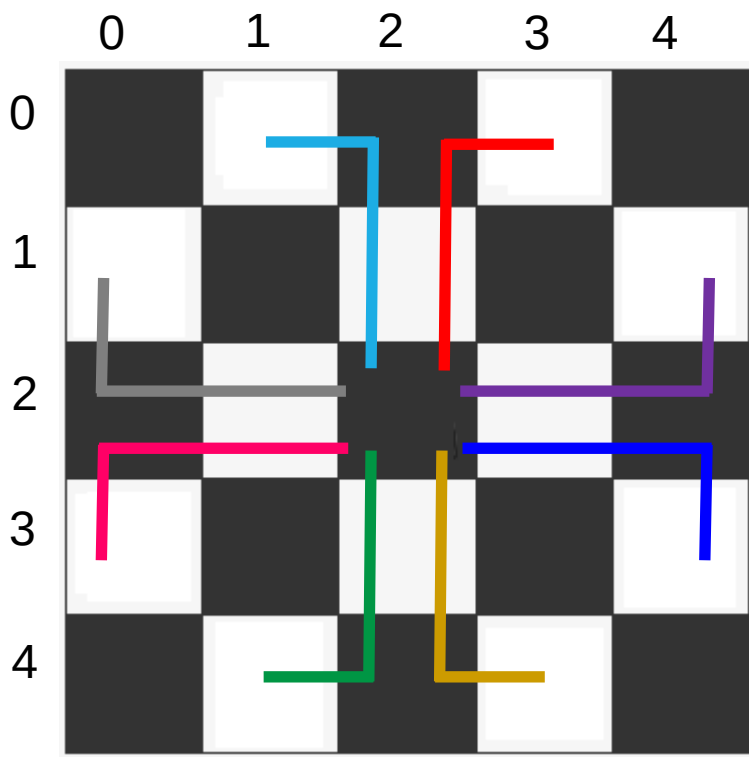


```
}
```

```
int solveKTUtil(int x, int y, int movei, int sol[N][N], int xMove[N], int yMove[N])  
{
```



Possibilidades de movimento  
do cavalo no tabuleiro



```
}
```

```

int solveKTUtil(int x, int y, int movei, int sol[N][N], int xMove[N], int yMove[N])
{
    if (movei == N * N)    // caso base: cavalo visitou todas as casas
        return 1;

    for (int k = 0; k < 8; k++)    // pega os movimentos certos do cavalo
    {
        int next_x = x + xMove[k]; // { +2, +1, -1, -2, -2, -1, +1, +2 }
        int next_y = y + yMove[k]; // { +1, +2, +2, +1, -1, -2, -2, -1 }

        if (isSafe(next_x, next_y, sol))    // analisa se o movimento do cavalo é seguro
        {
            sol[next_x][next_y] = movei;
            if (solveKTUtil(next_x, next_y, movei + 1, sol, xMove, yMove) == 1)
                return 1;    // solução encontrada
            else    // beco sem saída
                sol[next_x][next_y] = -1;    // backtracking
        }
    }
    return 0;
}

```

```

int isSafe(int x, int y, int sol[N][N]){
    return (x >= 0 && x < N &&
            y >= 0 && y < N &&
            sol[x][y] == -1);
}

```

# PARA PRATICAR...

- Implemente o código mostrado em aula para o problema do passeio do cavalo na linguagem C.

|           |
|-----------|
| main()    |
| solveKT() |

|                                                      |
|------------------------------------------------------|
| solveKT()                                            |
| ...<br>solveKTUtil(...)<br>...<br>printSolution(...) |

|                    |
|--------------------|
| printSolution(...) |
| ...                |

|                                       |
|---------------------------------------|
| solveKTUtil(...)                      |
| isSafe(...)<br>...<br>// backtracking |

|             |
|-------------|
| isSafe(...) |
| ...         |